

Predictive Modeling for Product Pricing on Amazon

Imran Mohammed, Amogh Vig. Sruthi Pappu

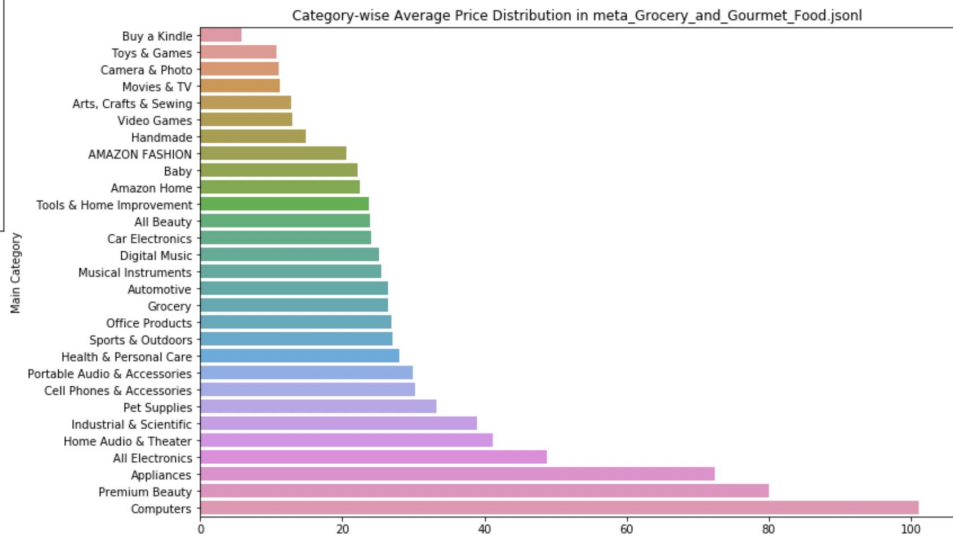
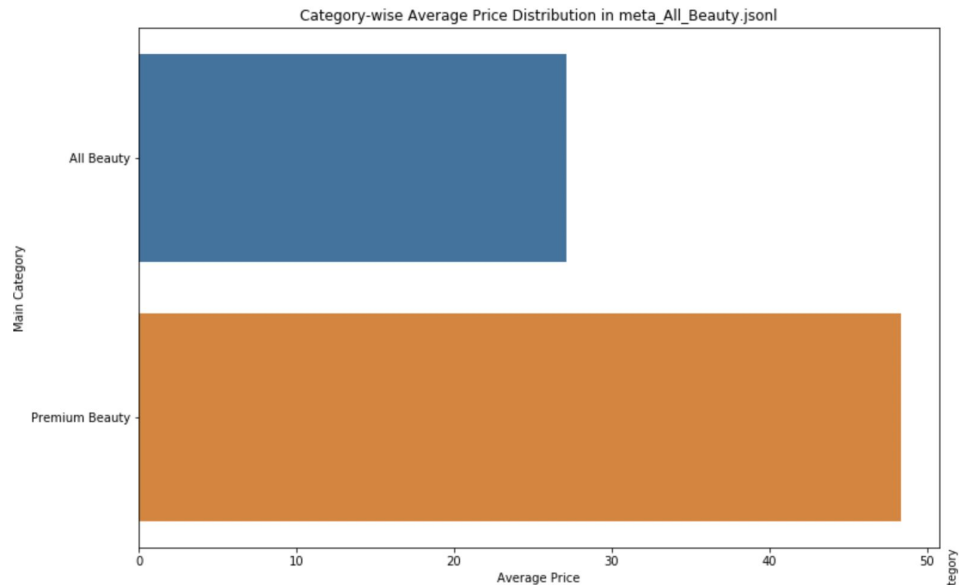
Outline

- Project Description
- EDA
- Pipelines Used
- Conclusion and Next Steps
- Issues/Problems

Project Description

- The goal is to develop a machine learning model to predict the price of products listed on Amazon.
- The model will utilize features such as category, brand, and specifications to make predictions.
- Regression techniques like linear regression, random forest, and XGBoost will be employed.
- The project aims to provide insights into pricing strategies for sellers and assist buyers in making informed decisions.
- The ultimate objective is to optimize pricing for both sellers and buyers, improving the overall e-commerce experience.

EDA



EDA

```
root
|-- author: struct (nullable = true)
|   |-- about: array (nullable = true)
|   |   |-- element: string (containsNull = true)
|   |-- avatar: string (nullable = true)
|   |-- name: string (nullable = true)
|-- average_rating: double (nullable = true)
|-- bought_together: string (nullable = true)
|-- categories: array (nullable = true)
|   |-- element: string (containsNull = true)
|-- description: array (nullable = true)
|   |-- element: string (containsNull = true)
|-- details: struct (nullable = true)
|   |-- 1 star: string (nullable = true)
|   |-- 15 99: string (nullable = true)
|   |-- 2 star: string (nullable = true)
|   |-- 22 99: string (nullable = true)
|   |-- 3 star: string (nullable = true)
|   |-- 4 star: string (nullable = true)
|   |-- 467 76: string (nullable = true)
|   |-- 5 star: string (nullable = true)
|   |-- ABPA Partslink Number: string (nullable = true)
|   |-- AC Adapter Current: string (nullable = true)
|   |-- ASTM Fluid Rating: string (nullable = true)
|   |-- Access Location: string (nullable = true)
|   |-- Accessory Connection Type: string (nullable = true)
|   |-- Action: string (nullable = true)
|   |-- Active Ingredients: string (nullable = true)
|   |-- Active Surface Area: string (nullable = true)
|   |-- Actors: string (nullable = true)
|   |-- Actuator Type: string (nullable = true)
```



Pipelines Used

- First used a pipeline to apply one shot encoding onto some classifying string features
- Later used a basic assembler + model pipeline for the training
- Note:
 - We also used Hashing-TF and IDF on some string text features like “title”
 - Didn't use a pipeline, we ran the model on it

```
from pyspark.sql.functions import concat_ws
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml import Pipeline

# Ensure this step is applied correctly before defining the pipeline stages
df_filtered = df_filtered.withColumn("categories_str", concat_ws(", ", "categories"))
# df_filtered.cache()

indexers = [
    StringIndexer(inputCol=column, outputCol=column + "_Index", handleInvalid="keep")
    for column in ['main_category', 'store', 'categories_str']
]

# Define OneHotEncoder for the indexed columns
encoders = [
    OneHotEncoder(inputCol=indexer.getOutputCol(), outputCol=indexer.getInputCol() + "_Vec")
    for indexer in indexers
]

# Pipeline stages
stages = indexers + encoders

# Define the pipeline
pipeline = Pipeline(stages=stages)

# Fit and Transform
model = pipeline.fit(df_filtered)
df_transformed = model.transform(df_filtered)
```

Pipelines Used

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml import Pipeline

# Assemble features into a single vector
featureCols = ["main_category_Vec", "store_Vec", "categories_str_Vec", "average_rating", "rating_number", "title_features", "features_features"]
assembler = VectorAssembler(inputCols=featureCols, outputCol="features")

# Linear Regression Model
lr = LinearRegression(featuresCol="features", labelCol="price")
stages += [lr]
|

# Add the assembler to the stages of your pipeline
stages = [assembler, lr]
```


Pipelines Used



```
# Random Forest
from pyspark.ml.regression import RandomForestRegressor
|
# Define the Random Forest model
rf = RandomForestRegressor(featuresCol="features", labelCol="price")

# Reuse the assembler from previous steps and create the stages array
stages = [assembler, rf] # Use the same feature assembler as before
```

Conclusion and Next Steps

- We have some preliminary results for two regression models: linear and random forest
 - The linear model has a train RMSE of 207 and a test RMSE of 137
 - The random forest model has a train RMSE of 230 and a test RMSE of 180
- Thus, the linear model is performing slightly better than random forest
- For next steps, we will:
 - Implement other models, such as XGBoost
 - Tune the parameters of our model to improve performance
 - Add regularization to our pipeline to reduce model error
 - Normalize our data

Open Issues

- Our models exhibit both high train and test error
 - Our strategies to address this were discussed on the previous slide
- GCP has a hard time running all the code chunks from start to finish
 - We need to implement more checkpoints to avoid running the same code cells again (we can start the notebook halfway through)